

A Dataflow Framework for Structured Memory and Learning in Soar

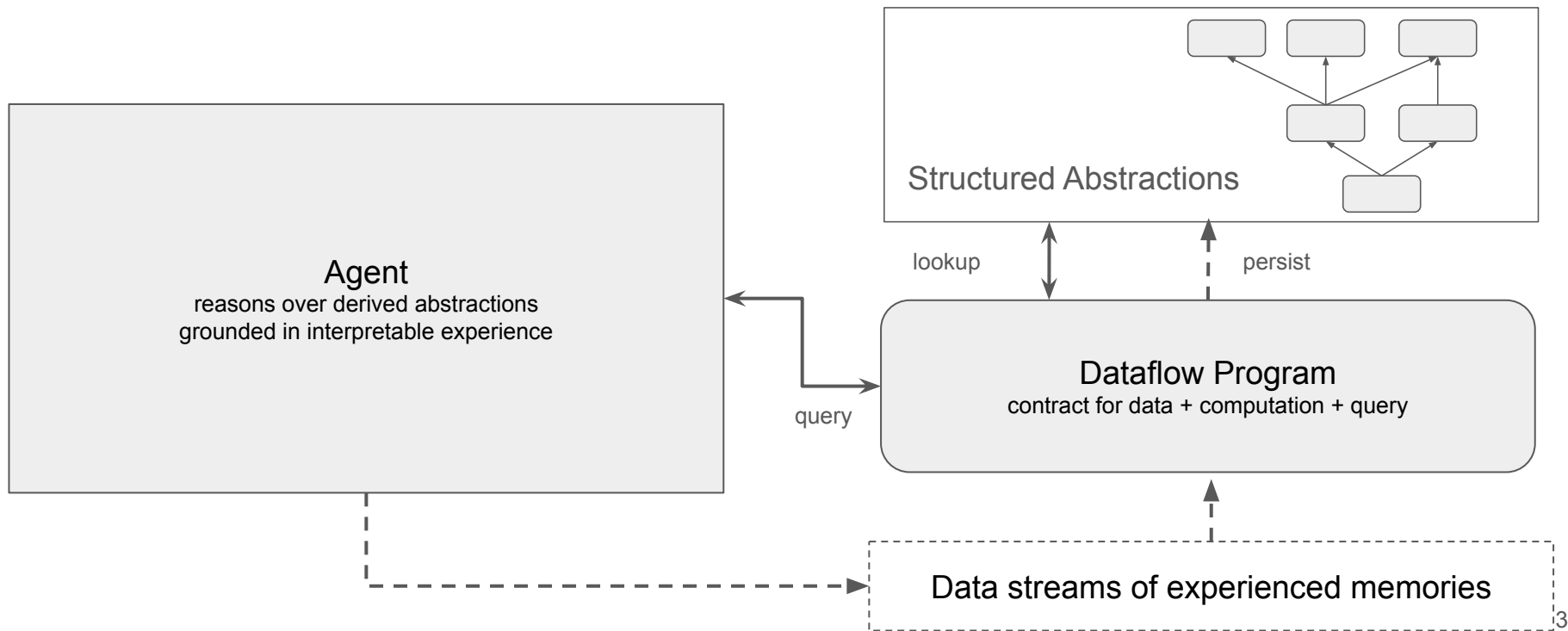
Soar Workshop 2026
Yongjia Wang

Motivation: Expanding Computation Over Memory

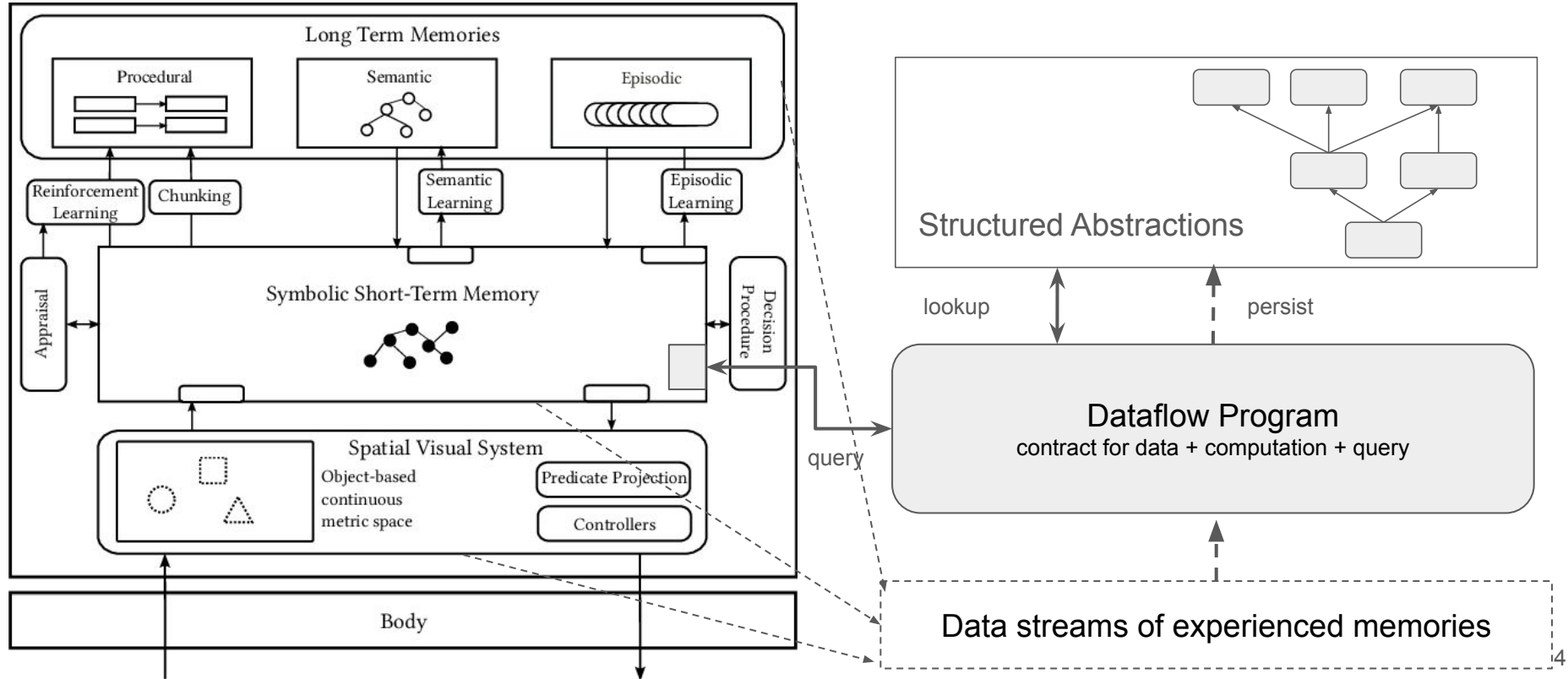
- Declarative learning requires exploring richer representations and computation (e.g., modalities, aggregation, statistical learning)
- These are tightly coupled to system implementation, making systematic exploration difficult
- **We introduce a dataflow-based framework that separates the definition of structure and computation from orchestration, enabling new representations and computations to be explored without redesigning the system**
 - Inspired by modern dataflow systems (Spark, Flink, Beam)

Dataflow Program for Structured Abstractions

A single program defines structure, computation, and query over collections of data



Extending Soar Memory with Dataflow Program



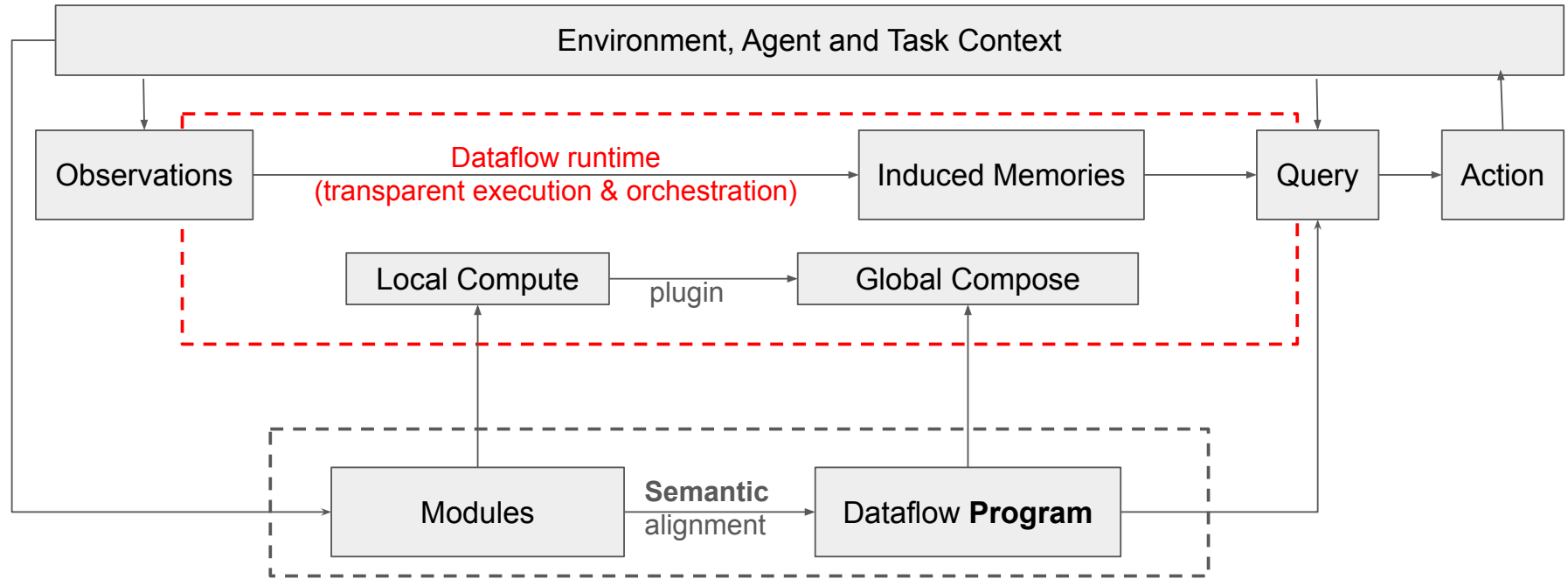
New Memory Infrastructure Capabilities

- Soar today: store and match instances
- This work: enable learning structured abstractions with pluggable computations

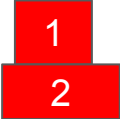
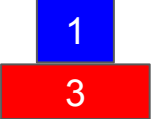
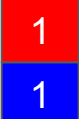
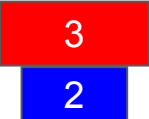
Declarative Memory Representation and Learning	Soar Today	This Work
Record, match and retrieve instances	Y	Y
Aggregate instances into compact abstractions	N	Pluggable computations (demo)
Compose computations across modalities	Limited	
Adapt to new symbols and structure hypothesis	Limited	Program Synthesis (future work)

The “Semantic Program” Core Abstraction

- Dataflow program composed of semantically aligned computational modules
- Defines structured computation over memory (analogous to productions in Soar)

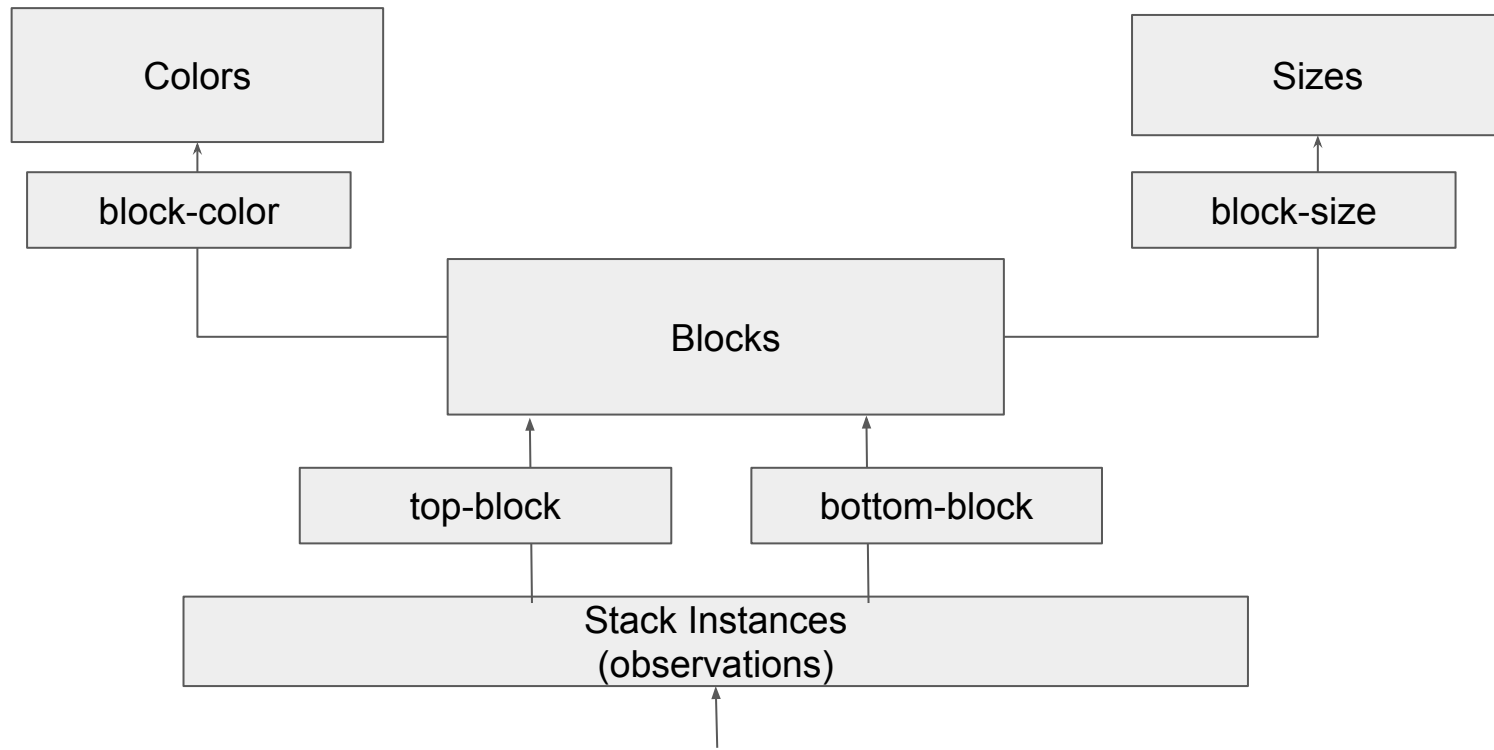


Blocks World Reference Manual (used throughout demo)

Symbolic Dimensions	Symbolic Values, Modal Representations, Constraints	
Color	RED 	BLUE 
Density	1	2
Size	SIZE_1  SIZE_2 	SIZE_3 
Stack Position	TOP 	BOTTOM 
Blocks (Color+Size)	     	
Stack instances (top Block on bottom Block) Weight=Density*Size	Weight(TOP) < Weight(BOTTOM) (14 out of 36 valid stacks)  1/2  2/3  2/3  1/2  3/4	

Semantic Program Structure for the Domain

Incremental and continuous learning of statistical regularities with fixed program

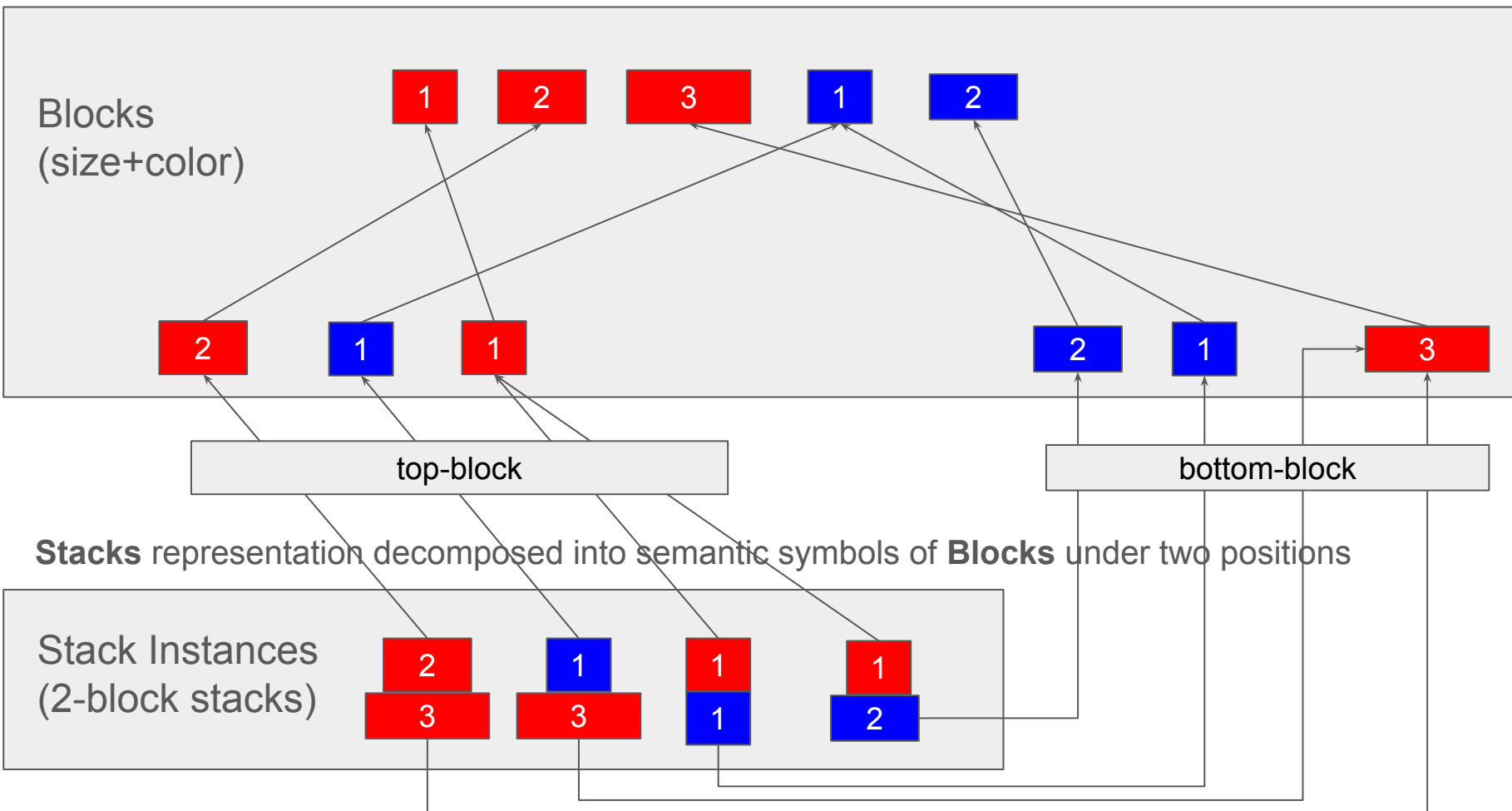


Reusable and Composable Modules

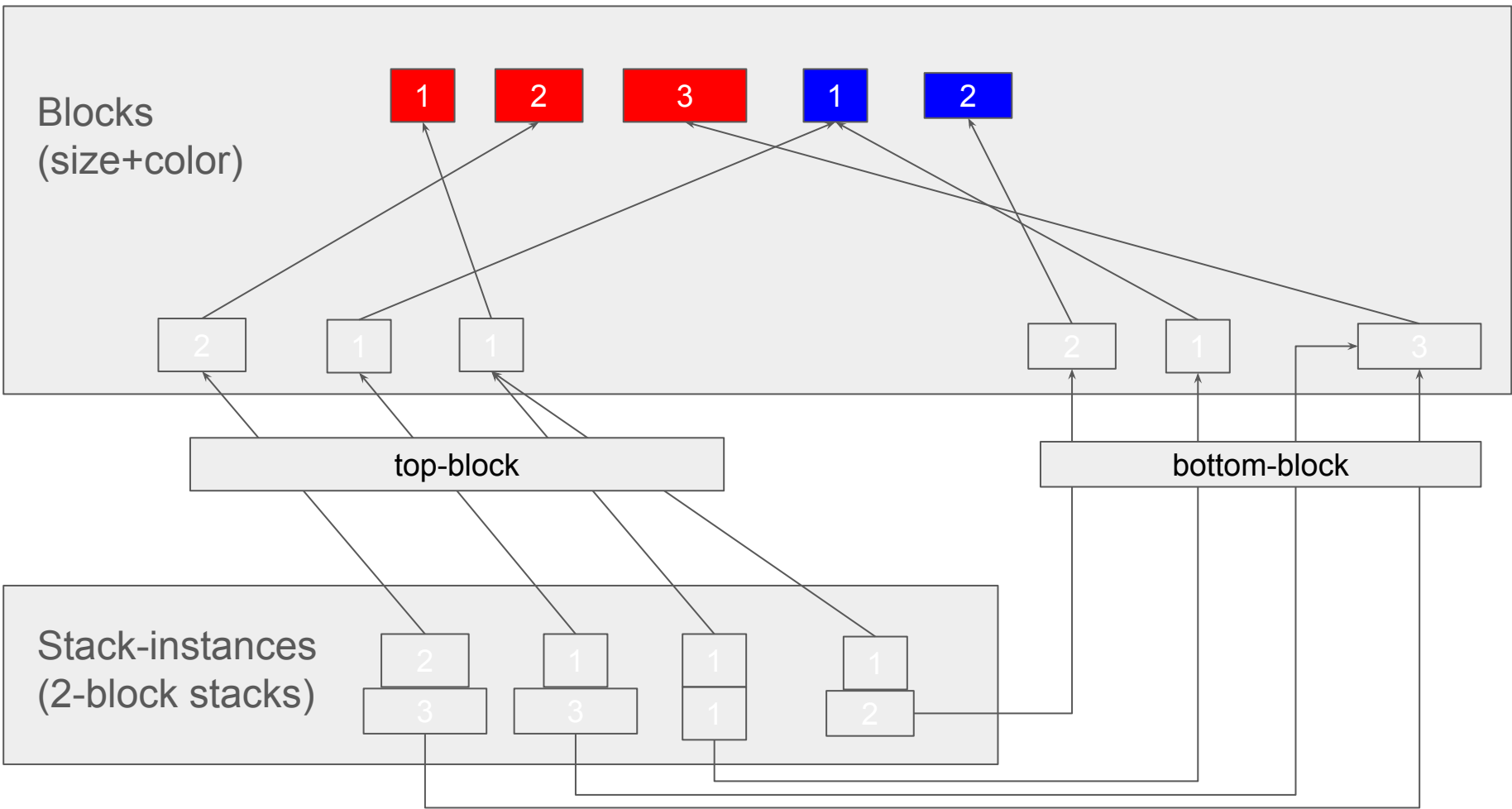
- defines computations and representations

Modules	Symbolic Dimensions	Symbolic Values and Functions
Visual Perceptual System	Color	RED, BLUE
	Size	SIZE_1, SIZE_2, SIZE_3
	Block (Color + Size)	(color=RED, size=SIZE_1), ...
	Stack (Block on Block)	(top=block1, bottom=block2), ...
Weight sensor	Weight	Numeric units: 1,2,3,4,5,6,...
Math & Physics Knowledge Functions & Predicates	density=weight/size weight=density*size numeric-compare: $x < y = ?$	(size=1, weight=1) -> density=1 (size=2, weight=4) -> density=2

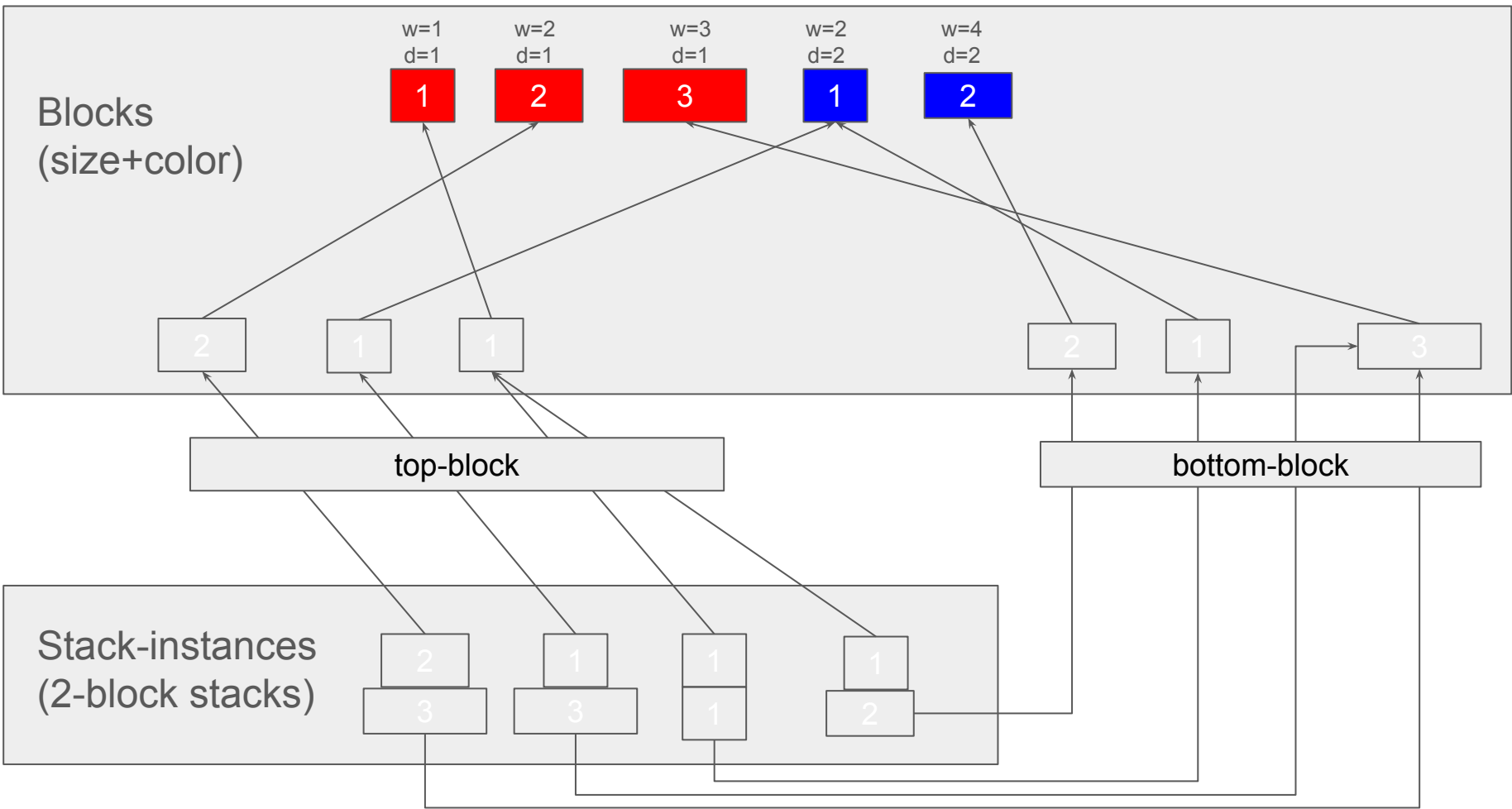
Merge **Block variants** with the same **symbolic reference** (addressed by size+color)



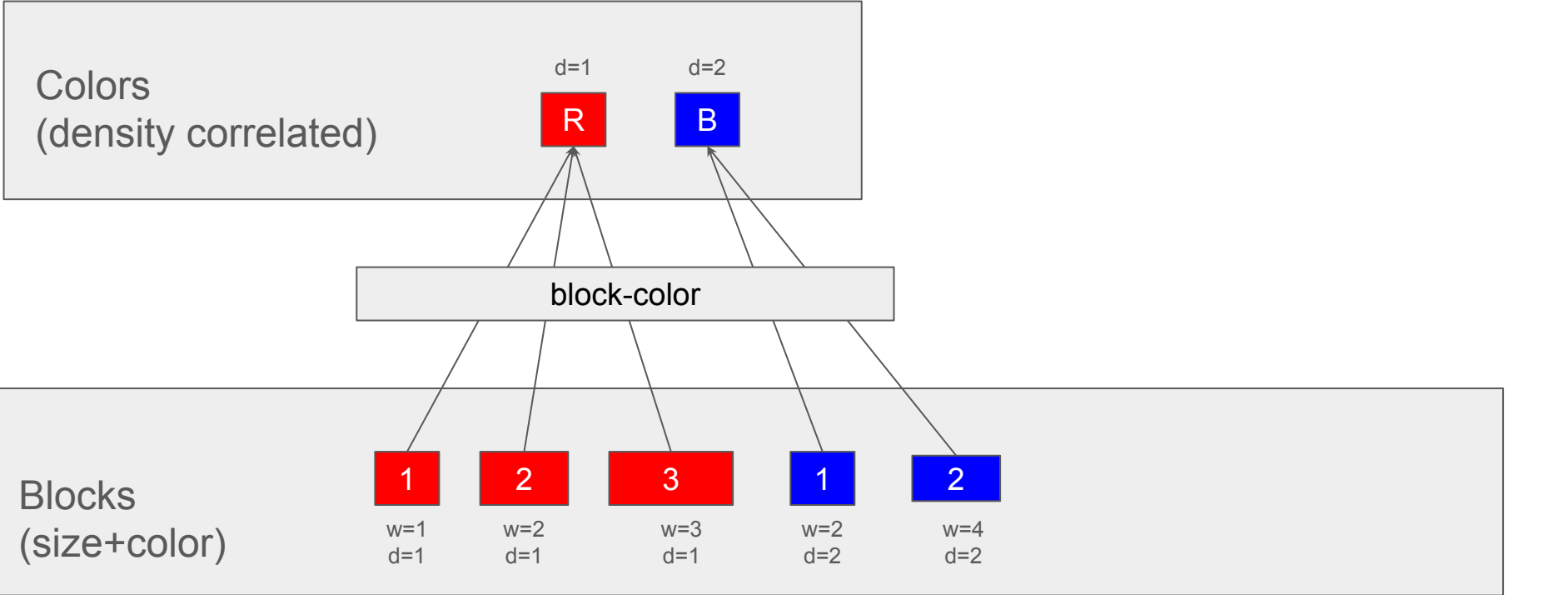
Representations consolidated by minimizing **residuals**, symbols and references persist.



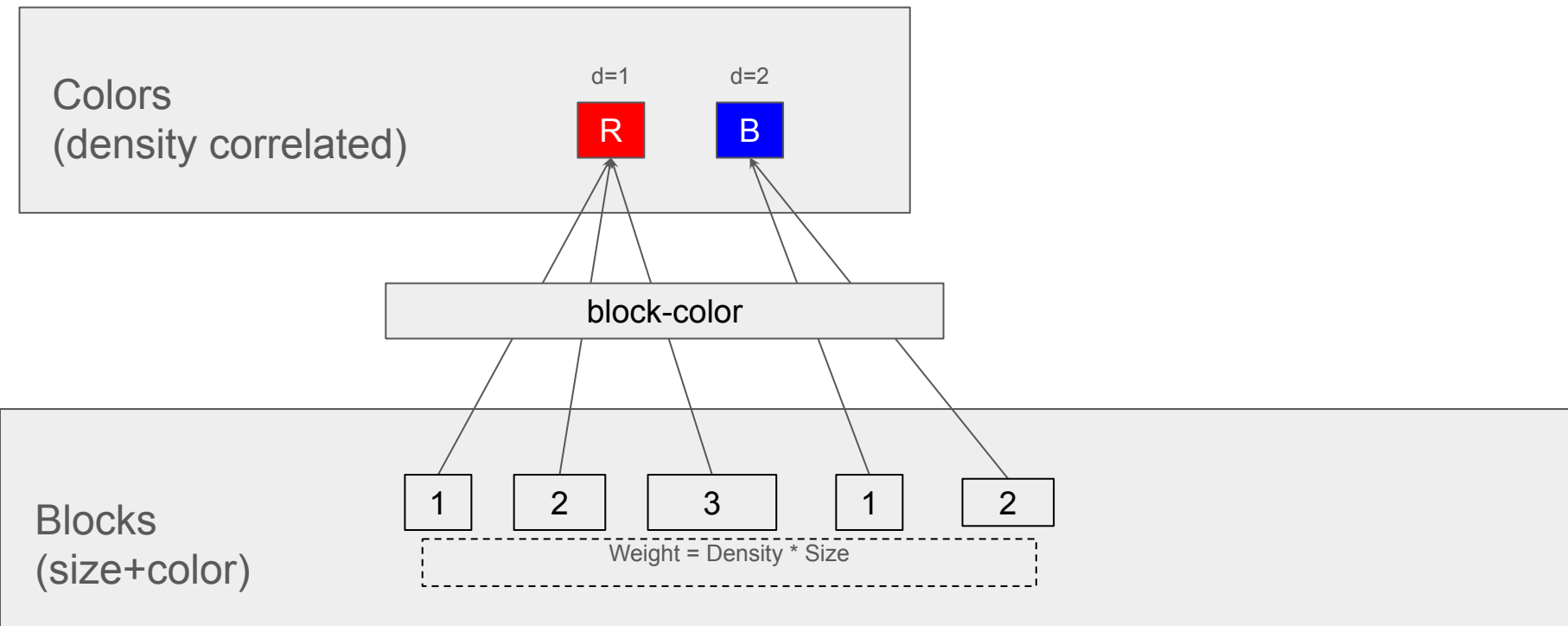
Add Weight(w) and Density(d) as additional properties



Color representation variants merge by Color symbol - density is associated with Color

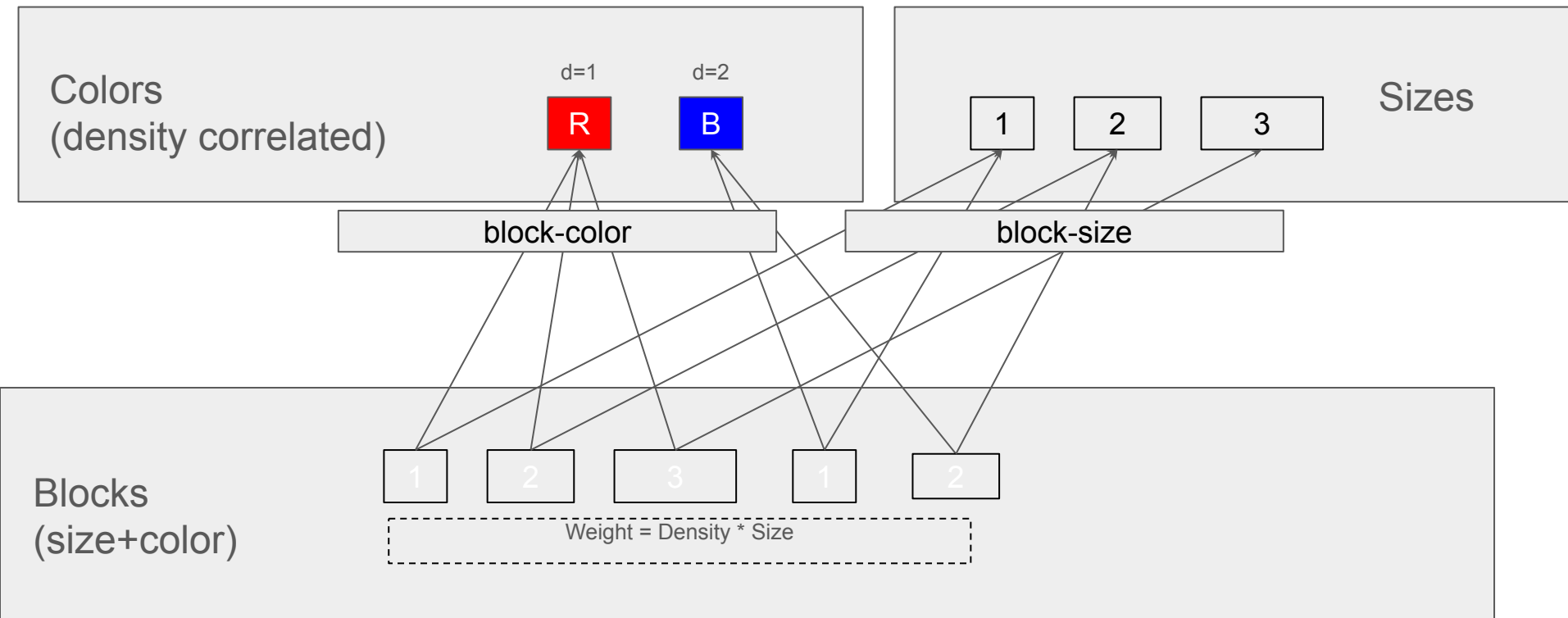


Colors representation consolidated with Colors symbol



Sizes residuals stay with Blocks symbol due to high entropy with Colors symbol
Weight can be recreated from density and size (prior knowledge)

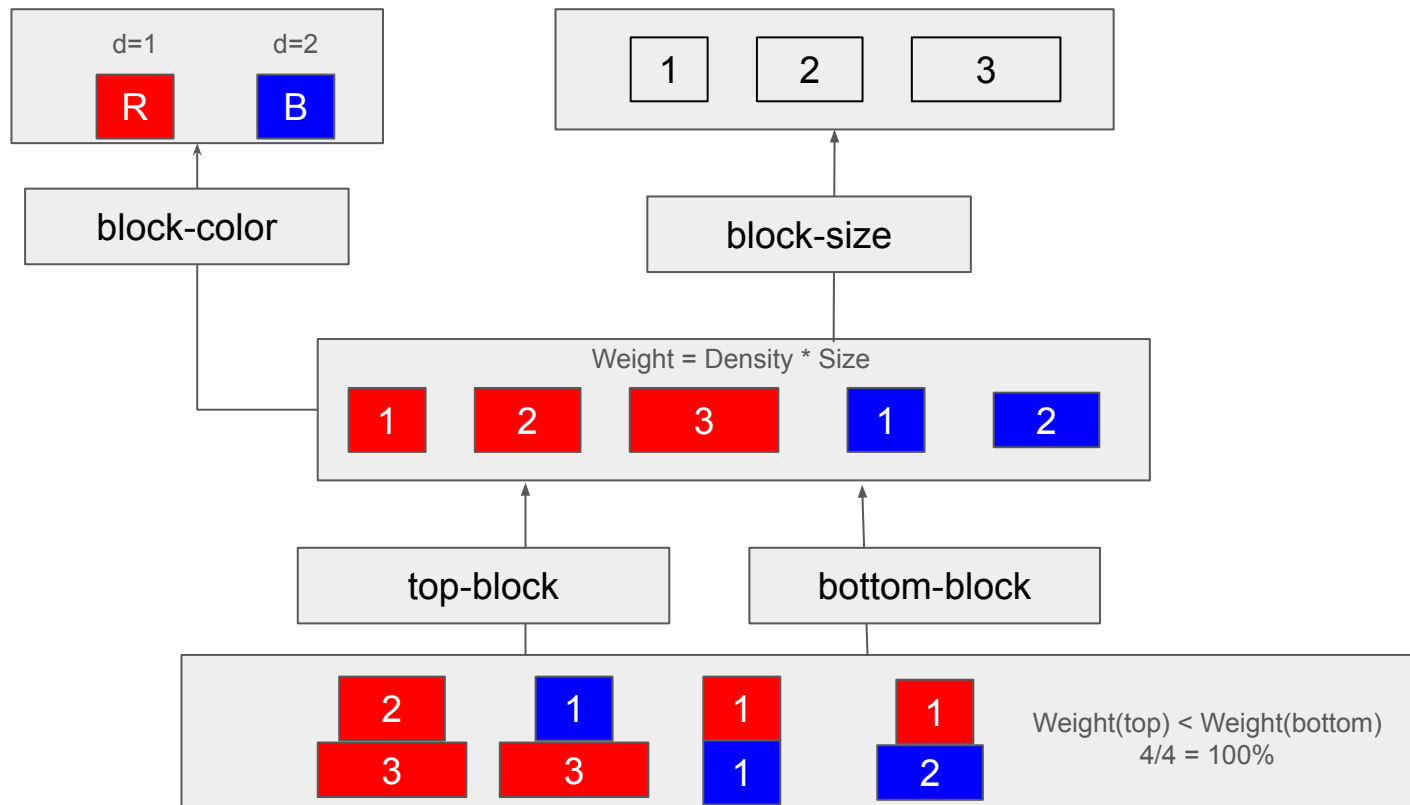
Sizes representation consolidated with Sizes symbol



No more residuals with Blocks

Induced Memory Representations

- Constraints are learned as aggregated regularities over structured representations



Semantic Query for Novel Inputs with Automatic Composition

Blocks:

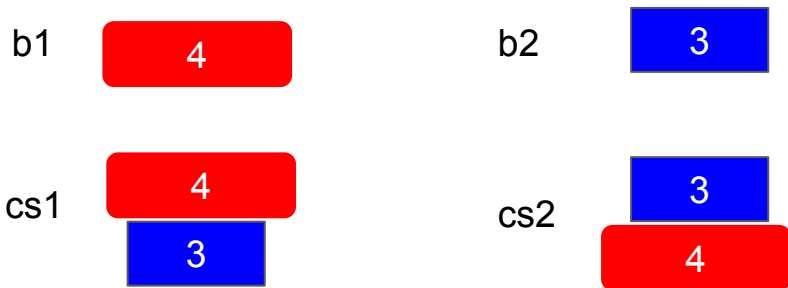
b1: Color=red, Size=4

b2: Color=blue, Size=3

Candidate Stacks:

cs1: top=b1, bottom=b2

cs2: top=b2, bottom=b1



Query:

Evaluate(StackInstances, cs1)

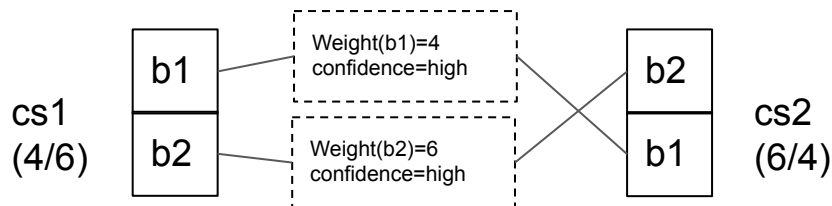
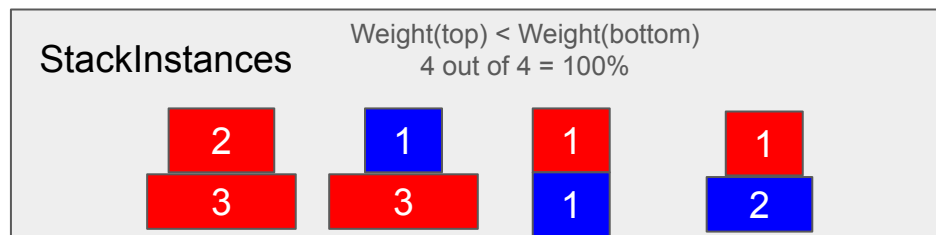
Evaluate(StackInstances, cs2)

Result:

cs1 → high novelty, **high** consistency, high confidence

cs2 → high novelty, **low** consistency, high confidence

Decision: prefer cs1



Semantic Query Evaluation Details

Query combines local computations with global composition over memory.

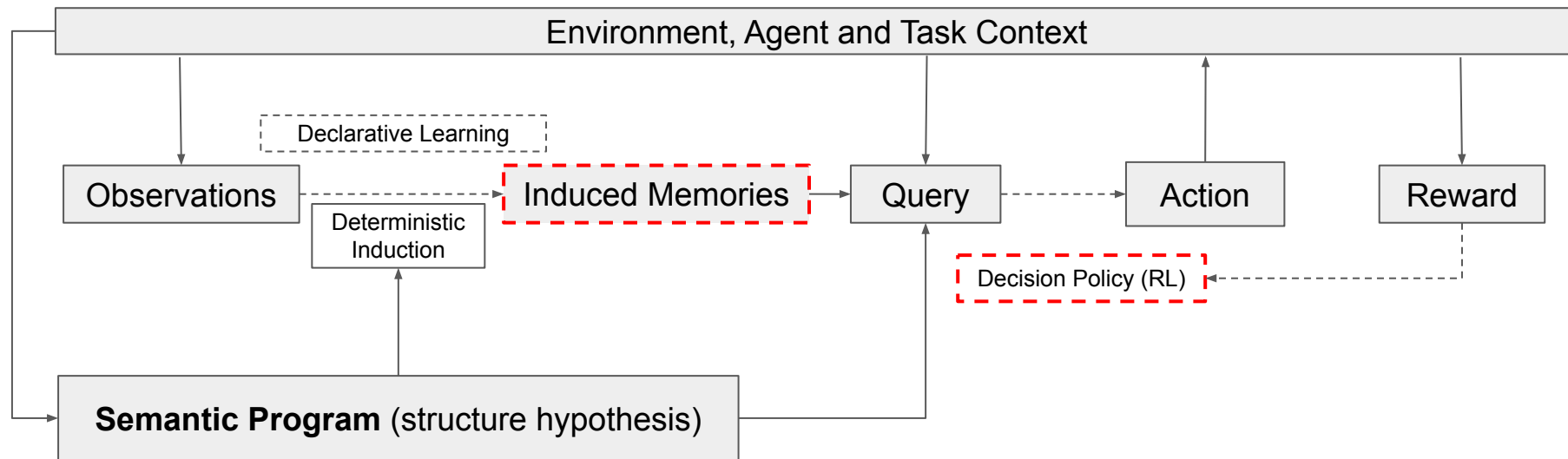
Query Input			Structured Evaluation Metrics for Agent Decision Making		
Dimension	Context	Value	Novelty (observation)	Consistency (constraint satisfaction)	Confidence (prediction)
StackInstance	cs1	top=b1, bottom=b2	High novelty	High consistency b1.weight < b2.weight (4/6)	confidence = 0.7 b1.Weight=4, confidence=0.8 b2.Weight=6, confidence=0.9
StackInstance	cs2	top=b2, bottom=b1	High novelty	Low consistency b2.weight > b1.weight (6/4)	
Block	b1	color=red, size=4	High novelty		Weight=4, confidence=0.8
Color	b1.color	red	Low novelty		Density=1, confidence=0.9
Size	b1.size	4	High novelty		
Block	b2	color=blue, size=3	Medium novelty		Weight=6, confidence=0.9
Color	b2.color	blue	Low novelty		Density=2, confidence=0.9
Size	b2.size	3	Low novelty		18

Benefits for Soar Agents

- Reduces production rule complexity
 - Encapsulates heterogeneous computations—perception, aggregation, prediction, materialization, compression, and query—within structured representations
- Enables reusable domain structures (modules)
- Supports principled exploration (novelty)
- Supports reliable decisions (consistency + confidence)

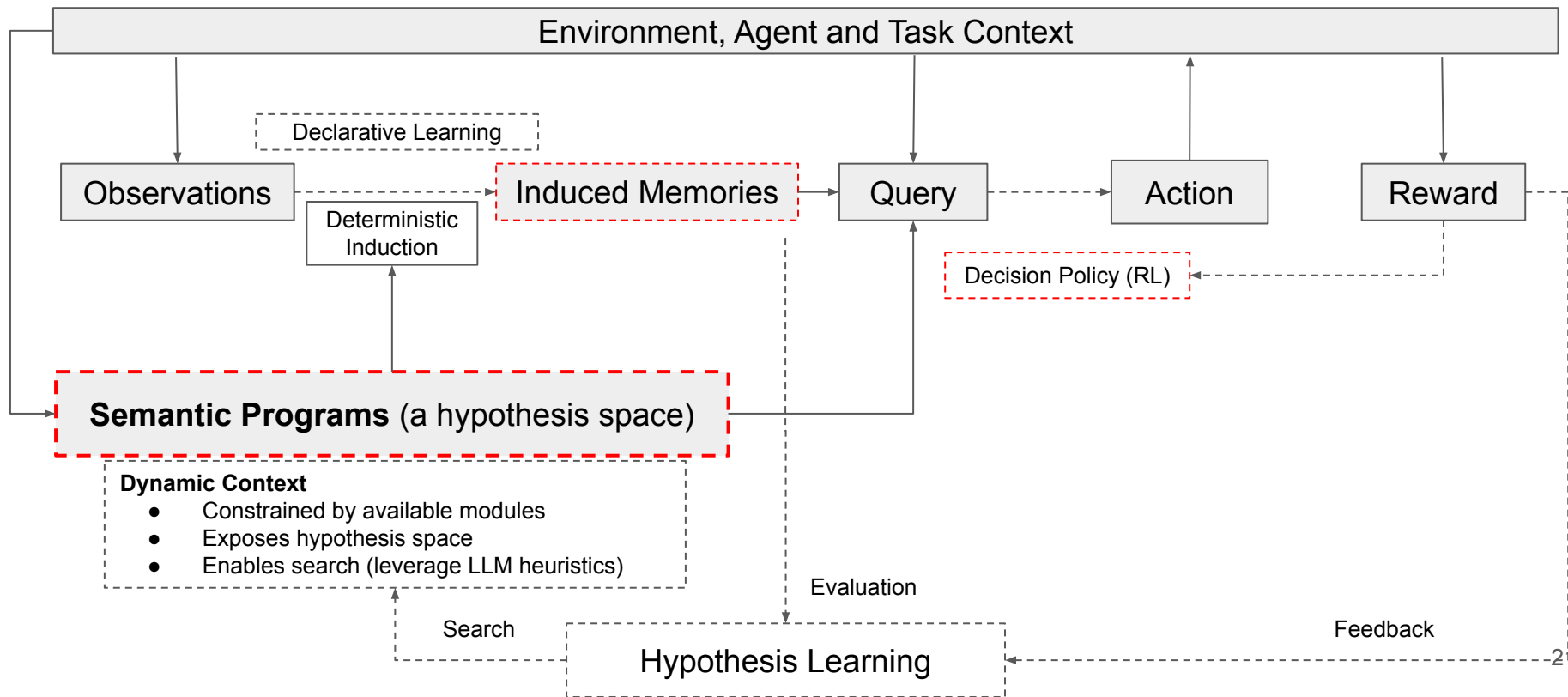
Integrate with Reinforcement Learning

- fixed program defines how memory is structured and how computations are composed



Future Work

The program itself is the target of structure hypothesis learning



Nuggets and Coal

- Nuggets
 - Semantic program abstraction
 - Encapsulation of data/memory, computation, and query (inherits dataflow)
 - Semantic alignment and closure (beyond dataflow)
 - Enable learning of structured abstractions from experience
 - Make symbolic structure learnable
- Coal
 - Demonstrated in a controlled proof-of-concept setting
 - Learning programs is not yet realized
 - Formal objectives and efficient search for structure learning remain open questions

Code & materials: github.com/yongjiaw/semantic-program

yongjiaw@gmail.com